Otimização de armazenamento de recursos em jogos de sobrevivência

Restrições de compatibilidade e capacidade | SME 0110 - Programação Matemática

Relatório técnico de implementação e experimentos. Versão de 04/11/2026. Cenário fictício e dados sintéticos. Autoria e composição do grupo devem ser identificadas pelos integrantes antes da submissão.

Situação da entrega: código e validação implementados; Gurobi e SCIP executados em instâncias de 20 itens; licença Gurobi disponível impede os experimentos maiores. O relatório distingue execuções concluídas, limites de tempo e indisponibilidades. A comparação integral dos quatro tamanhos ainda depende de licença adequada. A aceitação do cenário fictício como aplicação prática deve ser confirmada com o professor.

1. Introdução

Jogadores de sobrevivência acumulam madeira, pedra, metal, alimentos, poções e materiais perigosos. Organizar esses recursos em um inventário limitado de baús é uma aplicação didática de alocação: todos os itens precisam ser guardados, e certos pares não podem coexistir. A tarefa é minimizar a quantidade de baús utilizados, respeitando capacidade escalar e incompatibilidades. Isso difere de uma mochila de seleção, na qual seria permitido descartar itens.

O cenário foi proposto pelo grupo e não corresponde a regras ou dados coletados de um jogo comercial. A analogia computacional é a consolidação de materiais em recipientes limitados, com separação de classes incompatíveis. A implementação pode apoiar o planejamento de inventário de um jogo ou servir de protótipo para alocação logística, mas a transferência industrial exigiria validar unidades, restrições de segurança e dados reais.

O Bin Packing com conflitos foi estudado por Muritiba et al. [1], combinando restrições de capacidade com incompatibilidades entre itens. Dósa [2] apresenta a garantia de aproximação do FFD clássico, que ordena itens por tamanho antes de aplicar First Fit. Essa garantia não é assumida aqui: estoque finito, capacidades diferentes e conflitos alteram o problema. Bacci e Nicoloso [3] mostram que a estrutura do grafo de conflitos pode afetar a dificuldade dos benchmarks; por isso, uma única família de conflitos por categoria não sustenta conclusões gerais de desempenho.

A base didática é o PDF do projeto e os materiais do Archive.zip [6]: Aula 05 fornece o modelo de empacotamento; Aula 06 e o notebook de dimensionamento de lotes exemplificam vínculos entre decisões e ativação; Aula 08 fundamenta a alocação como partição. Os materiais de Simplex ajudam a interpretar relaxações e limites, mas não substituem a exigência de solução inteira e heurística.

2. Modelagem Matemática

2.1 Hipóteses, conjuntos e parâmetros

I é o conjunto de itens indivisíveis; J é o inventário finito de baús; E é o conjunto de pares não ordenados de itens incompatíveis. Cada item tem tamanho w_i inteiro positivo e categoria k_i; cada baú tem capacidade C_j inteira positiva. A unidade é UA (unidade abstrata de armazenamento), usada tanto em tamanhos quanto em capacidades. Não há encaixe geométrico, custos diferenciados nem elegibilidade por tipo de baú. Todas as entradas são validadas antes da solução.

As decisões operacionais adotadas nesta implementação são configuráveis e não representam regras previamente aprovadas pelo grupo: alimento e pocao conflitam com explosivo e combustivel. As demais combinações são permitidas. Os quatro pares de categorias geram conflitos entre todos os respectivos

itens; regras adicionais por ID também são aceitas. Uma regra de categoria consigo mesma impede dois itens dessa categoria de compartilharem um baú.

2.2 Variáveis, função objetivo e restrições

x_ij é binária e vale 1 quando o item i é alocado ao baú j. y_j é binária e vale 1 quando o baú j é utilizado. O mesmo modelo é construído nas APIs nativas de Gurobi [4] e PySCIPOpt [5]. A notação sum indica somatório sobre o conjunto especificado.

Família	Formulação	Interpretação
Objetivo	min sum(j em J) y_j	Menor número de baús
Alocação	sum(j em J) x_ij = 1, para cada i	Todo item exatamente uma vez
Capacidade	sum(i em I) w_i x_ij <= C_j y_j, para cada j	Consumo limitado à capacidade ativa
Vínculo	x_ij <= y_j, para cada i,j	Item só entra em baú ativo
Conflitos	x_ij + x_hj <= 1, para cada {i,h} em E e j	Pares incompatíveis separados
Não vazio	y_j <= sum(i em I) x_ij, para cada j	Baú ativo contém item
Domínio	x_ij,y_j em {0,1}	Decisões binárias

O vínculo explícito reforça a relaxação. A restrição de não vazio não muda o ótimo, mas impede baús ativos vazios em incumbentes. Se w_i>C_j, x_ij é fixada em zero. A simetria y_anterior >= y_seguinte é imposta somente entre baús de capacidade idêntica, que são intercambiáveis sob as hipóteses adotadas. Não se impõe essa ordenação entre capacidades distintas.

Antes da simetria, são n*m+m variáveis e n+2*m+n*m+|E|*m restrições, sem contar limites de variáveis. O crescimento dos conflitos pode tornar o modelo compacto caro mesmo com poucas categorias. Não é necessário um Big-M arbitrário: C_j é o limite físico.

2.3 Redução de candidatos e limites

O inventário original dos experimentos tem m=n. Se FFD produz solução completa com U baús, preservam-se apenas os U maiores baús para a otimização. Prova: OPT<=U; ordene as capacidades dos k<=U baús de uma solução ótima. A r-ésima maior capacidade global é pelo menos tão grande quanto a r-ésima dessa solução. Transferir cada grupo de itens para os k maiores baús mantém capacidades e conflitos, pois os grupos são preservados. Assim existe ótimo no inventário reduzido. A operação é idêntica nos dois solvers; se FFD falha, não se reduz J. A opção --no-reduction desativa a transformação.

A prova depende de a compatibilidade ser exclusivamente entre itens, sem custo ou restrição por tipo de recipiente. Selecionar arbitrariamente U baús, ou aplicar a redução a uma extensão com baús especializados, não teria a mesma garantia.

O limite inferior de volume é o menor k cuja soma das k maiores capacidades cobre a soma dos tamanhos. Ele ignora indivisibilidade e conflitos, portanto pode ser fraco. Se não houver k, o inventário é insuficiente. Nos resultados dos solvers, LB significa o melhor bound global da busca, não necessariamente a relaxação linear inicial. Com incumbente UB, o gap comum é 100*(UB-LB)/max(abs(UB),1), em porcentagem. O gap nativo de cada solver é armazenado separadamente, em fração. Ausência de bound ou incumbente gera null, não zero.

3. Solução via Solvers de Otimização

3.1 Implementação, ambiente e protocolo

O pacote possui módulos separados para leitura, verificação, heurística, modelo exato, geração e CLI. Gurobi e SCIP recebem as mesmas entradas, restrições, redução e parâmetros comparáveis: uma thread, seed 0, tolerância de gap alvo zero e nenhum warm start. O limite do piloto é de 10 segundos de otimização por instância. Construção e extração são medidas fora desse limite. Os logs e saídas estruturadas estão em results/pilot; cada registro contém o SHA-256 da entrada.

Ambiente registrado: Python 3.12.14; Linux-6.18.57-x86_64-with-glibc2.39; processador exposto como x86_64, 9 CPUs lógicas visíveis, uma thread por solver. Gurobi 13.0.3; PySCIPOpt 6.3.0; SCIP 10.0.2; NumPy 2.5.4. O ambiente virtualizado não expõe o modelo físico detalhado da CPU; não se assume equivalência com uma máquina local.

São 12 instâncias: n em {20,50,100,200} e sementes {11,22,33}. Tamanhos são amostrados uniformemente entre 1 e 9 UA; categorias, uniformemente entre sete opções. Há n baús: floor(n/10) de 20 UA, floor(n/5) de 15 UA e os demais de 10 UA. Como cada item cabe no menor baú, existe uma solução trivial com um item por baú. O estoque dos maiores é finito. As sementes representam instâncias diferentes, com apenas uma medição de cada método por instância, não repetições para estimar variância temporal.

3.2 Resultados do piloto

A tabela mostra objetivo e tempo total em segundos. Ótimos dos solvers têm gap comum de 0%. O traço significa objetivo inexistente. T = limite de tempo sem incumbente; L = indisponível por licença. A coluna FFD é solução factível, sem alegação geral de ótimo.

n / seed	FFD: baús	SCIP: baús / status	SCIP: total s	Gurobi: baús / status	Gurobi: total s
20 / 11	8	8 / ótimo	0.0328	8 / ótimo	0.0131
20 / 22	6	5 / ótimo	0.0197	5 / ótimo	0.0117
20 / 33	7	7 / ótimo	0.0311	7 / ótimo	0.0134
50 / 11	15	14 / ótimo	0.1907	- / L	0.0396
50 / 22	16	16 / ótimo	0.1911	- / L	0.0286
50 / 33	17	17 / ótimo	0.2065	- / L	0.0519
100 / 11	30	29 / ótimo	2.8015	- / L	0.1776
100 / 22	29	29 / ótimo	1.9539	- / L	0.1887
100 / 33	31	31 / ótimo	2.7787	- / L	0.2926
200 / 11	58	- / T	11.3700	- / L	1.4463
200 / 22	60	- / T	11.2809	- / L	1.2946
200 / 33	63	- / T	11.7442	- / L	1.4839

SCIP provou o ótimo nas nove instâncias com até 100 itens. Gurobi provou os mesmos objetivos nas três instâncias de 20 itens. Em 50, 100 e 200 itens, Gurobi encerrou com erro de licença 10010, “Model too large for size-limited license”. Esses tempos medem preparação e falha, não desempenho de otimização; não entram em uma classificação de rapidez. A comparação entre os solvers é válida apenas no subconjunto de 20 itens, no qual Gurobi apresentou menores tempos nesta execução única. Não se

conclui superioridade geral.

Em 200 itens, SCIP encerrou o piloto com status timelimit, sem incumbente e sem bound global finito registrado. Isso não significa infactibilidade: FFD fornece uma solução completa para cada entrada. O tempo total ultrapassa 10 s porque construção e extração não estão dentro do limite de otimização. Não há gap de solver definido sem solução inteira.

3.3 Validação e rastreabilidade

Foram aprovados 34 testes automatizados. Os testes incluem bordas (vazio, capacidade exata, item grande e estoque insuficiente), conflitos, categorias auto-incompatíveis, dados inválidos e soluções adulteradas. Doze pequenas instâncias aleatórias com cinco itens e três baús são resolvidas por enumeração independente das 3^5 alocações possíveis; cada solver é comparado a esse ótimo com e sem redução (48 comparações solver/oráculo). Isso testa a implementação e a transformação de candidatos, além de comparar os dois solvers entre si.

O verificador não consulta variáveis dos solvers: reconta IDs, cargas, pares de conflitos, baús e objetivo a partir da alocação final. A extração também verifica ativação y_j. Objetivo nunca é consultado sem incumbente. Estados de erro, licença, ausência de solução e infactibilidade provada permanecem distintos. Resultados sem solução não são certificados como factíveis. As saídas JSON preservam a alocação completa e os tempos separados de construção/otimização/total; summary.csv consolida as métricas.

3.4 Execução complementar de 200 itens

Após a ausência de incumbentes no piloto, repetiu-se somente SCIP em 200 itens com limite de 60 s, mantendo os demais parâmetros. Esta análise é separada do piloto de 10 s e não constitui comparação temporal com os outros métodos sob o mesmo orçamento. Resultados em results/extended.

Instância	Status	Objetivo	LB global	Gap %	Tempo total s
game_n200_s11	optimal	57	57.0	0.00	35.749
game_n200_s22	optimal	59	59.0	0.00	34.309
game_n200_s33	feasible	63	62.0	1.59	61.421

A rodada complementar provou os ótimos 57 e 59 nas sementes 11 e 22. Na semente 33, encontrou 63 baús e bound 62, com gap de 1,59%; o ótimo pode ser 62 ou 63. A execução de 60 s melhora o conhecimento sobre as instâncias, sem resolver a pendência de Gurobi.

4. Método de Solução Alternativo

4.1 FFD adaptado

A escolha deriva da simplicidade e rapidez do FFD clássico [2], adaptado para as mesmas regras dos solvers. Primeiro ordenar os itens por tamanho decrescente, com desempate pela posição original. Para cada item, percorrer baús abertos na ordem de abertura, verificando carga e incompatibilidade com todos os ocupantes. Usar o primeiro factível. Se nenhum servir, abrir o maior baú disponível que comporte o item; desempatar pela posição na entrada. Se nenhum puder abrir, retornar heuristic_failed e nenhuma solução parcial como limite superior.

Abrir os maiores primeiro evita consumir muitos recipientes pequenos, mas não garante êxito com inventário finito. Exemplo: itens [4,3,3] e baús [6,4], sem conflitos. FFD coloca 4 no baú 6 e 3 no baú 4, restando um 3 sem lugar. Existe solução com [3,3] no baú 6 e [4] no baú 4. Portanto falha gulosa não prova infactibilidade. Em baús uniformes de 10, [6,5,3,2,2,2] leva FFD a três recipientes, embora [6,2,2] e [5,3,2] provem ótimo dois.

Na implementação direta, ordenar custa O(n log n + m log m). As varreduras de baús e seus ocupantes totalizam O(n²+n*m) no pior caso, usando consulta de conflitos em conjunto hash. A preparação de E por categorias custa O(n²) e ocorre na leitura, antes da cronometragem dos algoritmos. Memória adicional é O(n+m+|E|). Não se atribui à variante o bound 11/9 do FFD clássico.

4.2 Qualidade e tempo observados

FFD produziu soluções válidas nas 12 instâncias. A comparação abaixo usa ótimo provado de SCIP (piloto até 100 itens; execução complementar quando disponível para 200). Quando não há ótimo, a referência é apenas o LB de volume. Excesso ao ótimo = 100*(H-OPT)/OPT; distância ao LB = 100*(H-LB)/LB. Estes números não são o gap nativo de solver.

n / seed	FFD H	Referência	Tipo	Excesso/distância %	Tempo FFD ms
20 / 11	8	8	ótimo	0.00	0.062
20 / 22	6	5	ótimo	20.00	0.055
20 / 33	7	7	ótimo	0.00	0.076
50 / 11	15	14	ótimo	7.14	0.108
50 / 22	16	16	ótimo	0.00	0.114
50 / 33	17	17	ótimo	0.00	0.114
100 / 11	30	29	ótimo	3.45	0.454
100 / 22	29	29	ótimo	0.00	0.258
100 / 33	31	31	ótimo	0.00	0.238
200 / 11	58	57	ótimo	1.75	3.121
200 / 22	60	59	ótimo	1.69	1.523
200 / 33	63	62	LB volume	1.61	0.904

No piloto, FFD coincide com o ótimo em seis das nove instâncias de até 100 itens. Perde um baú em 20/22, 50/11 e 100/11: excesso relativo de 20%, 7,14% e 3,45%, respectivamente. Em 200 itens, as soluções heurísticas estão um baú acima do limite de volume. Esses resultados indicam rapidez útil para interação em jogo, mas não garantem desempenho em inventários apertados ou grafos de conflito diferentes.

O tempo FFD observado é inferior a 4 ms nas 12 instâncias, incluindo ordenação e empacotamento. A validação final da heurística é posterior à medida, enquanto o total dos solvers inclui extração/validação; leitura e serialização ficam fora de ambos. A diferença de escopo é explicitada para evitar falsa precisão. Uma comparação estatística exigiria repetir medições, controlar aquecimento de bibliotecas e variar classes de instâncias. Sem warm start, o solver pode não achar incumbente em 10 s, mesmo existindo uma solução gulosa imediata.

5. Conclusão

A implementação concretiza o tema com um modelo binário coerente com as aulas, duas interfaces de solver, heurística adaptada e validação independente. No piloto, ambos os solvers concordam nos três casos de 20 itens; SCIP prova o ótimo em todos os casos de até 100. A extensão de 60 s prova dois ótimos adicionais em 200 itens e obtém uma solução factível no terceiro caso, com gap de 1,59%. A heurística é rápida e factível na família sintética usada, mas apresenta perdas conhecidas e pode falhar em outras entradas factíveis.

O principal limite experimental é a licença Gurobi restrita, que impede a comparação completa em 50, 100 e 200 itens. Não se substituiu esse requisito por outro solver. A entrega contém comando para repetir o experimento em ambiente licenciado, resultados brutos, fonte do relatório e checklist de atualização. Dados sintéticos e uma só família de categorias limitam a validade externa das conclusões; o tema fictício ainda exige alinhamento com a orientação de aplicação real do enunciado.

O aprendizado central é distinguir formulação de algoritmo, solução factível de prova de ótimo, falha gulosa de infactibilidade e bound global de relaxação inicial. A redução segura de candidatos mostra como explorar dominância sem alterar o problema. Como evolução, propõem-se warm start igual nos dois solvers, variáveis de presença de categoria para reduzir restrições, estoques mais apertados, conflitos individuais aleatórios e dados de uma aplicação validada. Recursos geométricos ou tipos de baú exigiriam rever a prova de redução.

Referências

[1] A. E. F. Muritiba, M. Iori, E. Malaguti e P. Toth, “Algorithms for the Bin Packing Problem with Conflicts,” INFORMS Journal on Computing, vol. 22, no. 3, pp. 401-415, 2010. DOI: 10.1287/ijoc.1090.0355. Consulta de metadados e resumo, 04 nov. 2026.

[2] G. Dósa, “The Tight Bound of First Fit Decreasing Bin-Packing Algorithm Is FFD(I) <= 11/9 OPT(I) + 6/9,” LNCS 4614, pp. 1-11, 2007. DOI: 10.1007/978-3-540-74450-4_1. Consulta de resumo e metadados, 04 nov. 2026; não se reproduz a prova integral.

[3] T. Bacci e S. Nicoloso, “On the benchmark instances for the Bin Packing with Conflicts,” arXiv:1706.03526v3, 2017. https://arxiv.org/abs/1706.03526v3. Consulta de resumo e metadados, 04 nov. 2026.

[4] Gurobi Optimization, “Python Model Class Reference” e “Model Attributes,” documentação oficial do Gurobi Optimizer. https://docs.gurobi.com/projects/optimizer/en/current/reference/python/model.html e https://docs.gurobi.com/projects/optimizer/en/current/reference/attributes/model.html. Acesso: 04 nov. 2026. Versão executada: 13.0.3.

[5] PySCIPOpt, “Model API” e “Introduction: Model Object,” documentação oficial. https://pyscipopt.readthedocs.io/en/stable/api/model.html e https://pyscipopt.readthedocs.io/en/stable/tutorials/model.html. Acesso: 04 nov. 2026. Versão executada: 6.3.0.

[6] SME 0110 - Programação Matemática, “Projeto de Otimização: O Problema do Empacotamento,” 2026, pp. 1-4; materiais de aula fornecidos em Archive.zip. Cópias preservadas em enunciado/ e contexto/. Repositório de apoio: https://github.com/caue-paiva/prog-mat, commit a2943f1ea499052a24b09e428c8ccc51956a7124, consultado em 04 nov. 2026.